НЕГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ ЧАСТНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ФИНАНСОВО-ПРОМЫШЛЕННЫЙ УНИВЕРСИТЕТ «СИНЕРГИЯ»

Кафедра Инновационных и цифровых технологий

ЛАБОРАТОРНАЯ РАБОТА № 3

Обучающийся Волков Илья Викторович

(ФИО) (подпись)

Преподаватель Сибирев Иван Валерьевич

(ФИО) (подпись)

Москва 2026 г.

СОДЕРЖАНИЕ

Введение 3

1. Понятие осознанной интеграции 3

2. Этап сборки (Build) 4

3. Инструменты сборки: Gradle, Bazel, Webpack 5

4. Этап тестирования (Test) 6

5. Пирамида тестирования 7

6. Инструменты тестирования: JUnit, Jest, Playwright 8

Заключение 9

Список использованных источников 9

ВВЕДЕНИЕ

Современная разработка программного обеспечения невозможна без чёткой организации процессов сборки и тестирования. Именно эти этапы превращают исходный код, написанный разработчиками, в готовый к эксплуатации продукт, обладающий необходимым качеством. В рамках методологии DevOps сборка и тестирование объединены в концепцию непрерывной интеграции, однако само по себе применение инструментов автоматизации не гарантирует успеха — требуется осознанный, продуманный подход к организации этих процессов.

Термин «осознанная интеграция» подчёркивает, что недостаточно просто настроить автоматический запуск тестов при каждом коммите. Команда должна понимать, какие именно проверки необходимы, в какой последовательности их выполнять, как обеспечить быструю обратную связь разработчикам и как интерпретировать результаты. Осознанность предполагает также разумный баланс между скоростью сборки и полнотой тестирования, между глубиной проверок и стоимостью их выполнения.

Цель данной работы — рассмотреть ключевые принципы и инструменты этапов сборки и тестирования как составных частей непрерывной интеграции в DevOps. В работе будут изучены понятие осознанной интеграции, процесс сборки программных продуктов с обзором популярных инструментов (Gradle, Bazel, Webpack), а также виды и уровни тестирования с рассмотрением фреймворков JUnit, Jest и Playwright.

1. ПОНЯТИЕ ОСОЗНАННОЙ ИНТЕГРАЦИИ

Непрерывная интеграция (Continuous Integration, CI) — это практика частого слияния изменений, вносимых разработчиками, с основной веткой репозитория, сопровождаемая автоматической сборкой и тестированием. Термин был введён Гради Бучем в 1991 году и получил широкое распространение благодаря экстремальному программированию (XP) в конце 1990-х годов.

Осознанная интеграция — это не просто техническая реализация CI, а продуманный подход к организации процессов сборки и тестирования, учитывающий специфику проекта, квалификацию команды и требования бизнеса. Её отличительными чертами являются:

- Быстрая обратная связь — разработчик должен узнавать о проблемах в течение минут, а не часов;
- Воспроизводимость — сборка одной и той же версии кода должна давать идентичный результат в любой среде;
- Доверие к тестам — зелёные тесты должны означать работоспособность кода, а красные — реальную проблему;
- Экономическая обоснованность — затраты на тесты не должны превышать пользу от них;
- Поддерживаемость — сборочный конвейер и тесты сами должны быть кодом, который легко изменять.

Этапы сборки и тестирования тесно связаны друг с другом и образуют единый процесс: сначала исходный код преобразуется в исполняемую форму, затем полученный артефакт проверяется на соответствие функциональным и нефункциональным требованиям. В петле DevOps эти этапы расположены последовательно и вместе составляют сердцевину непрерывной интеграции. Неудачное выполнение любого из них означает, что изменения не могут быть доставлены пользователям, что превращает качество сборки и тестирования в критически важный фактор скорости всей команды.

2. ЭТАП СБОРКИ (BUILD)

Сборка — процесс преобразования исходного кода и сопутствующих ресурсов в артефакт, пригодный для исполнения или развёртывания. В зависимости от языка программирования и типа проекта результатом сборки может быть исполняемый бинарный файл, JAR- или WAR-архив, Docker-образ, бандл JavaScript или набор статических файлов веб-приложения.

Типовой процесс сборки включает следующие шаги:

- Разрешение зависимостей — загрузка внешних библиотек из репозиториев (Maven Central, npm, PyPI);
- Компиляция — преобразование исходного кода в машинный код или байт-код;
- Обработка ресурсов — оптимизация изображений, минификация CSS и JavaScript, обработка шаблонов;
- Линковка — объединение скомпилированных модулей в единый артефакт;
- Упаковка — создание финального дистрибутива (архива, контейнерного образа);
- Публикация — размещение артефакта в хранилище (Nexus, Artifactory, Docker Registry).

Современные системы сборки работают инкрементально — при повторном запуске они выполняют только те шаги, результаты которых устарели из-за изменений в исходном коде. Это достигается за счёт анализа графа зависимостей между задачами и использования кеширования промежуточных результатов. На крупных проектах такая оптимизация сокращает время сборки в десятки раз.

Воспроизводимая сборка (reproducible build) — одно из ключевых требований современной разработки. Одна и та же версия исходного кода должна давать идентичные байт-в-байт артефакты независимо от того, кто, когда и на какой машине производит сборку. Это требование важно для безопасности (проверка целостности поставок), отладки и юридической защиты.

Роль систем контроля версий в процессе сборки трудно переоценить. Платформы GitHub и GitLab не только хранят исходный код, но и выступают триггерами для автоматических сборочных конвейеров: каждый push или

создание merge request инициирует запуск соответствующего пайплайна. Встроенные в эти платформы системы CI (GitHub Actions, GitLab CI/CD) позволяют описать процесс сборки декларативно в YAML-файле и хранить его рядом с кодом.

3. ИНСТРУМЕНТЫ СБОРКИ: GRADLE, BAZEL, WEBPACK

На рынке существует множество систем сборки, каждая из которых имеет свою нишу применения. Рассмотрим наиболее востребованные из них.

Gradle — гибкая система сборки, созданная в 2007 году и ставшая основным инструментом для Android-разработки после её официального принятия Google в 2013 году. Gradle использует декларативный DSL на языках Groovy или Kotlin, что делает конфигурацию одновременно читаемой и программируемой. Ключевыми преимуществами Gradle являются инкрементальная сборка, распределённое кеширование через Gradle Build Cache и параллельное выполнение независимых задач.

Пример простейшего build.gradle.kts для Java-проекта:

```
plugins {
    java
    application
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.slf4j:slf4j-api:2.0.9")
    testImplementation("org.junit.jupiter:junit-jupiter:5.10.0")
}
```

```
application {
    mainClass.set("com.example.Main")
}
```

Bazel — система сборки, разработанная Google на основе внутреннего инструмента Blaze и выпущенная в открытом доступе в 2015 году. Bazel ориентирован на крупные монорепозитории, содержащие код на множестве языков программирования. Его главная особенность — герметичные воспроизводимые сборки: все зависимости явно декларируются, а процесс изолирован от окружения. Это обеспечивает максимальную воспроизводимость и эффективное распределённое кеширование.

Webpack — самый известный бандлер для JavaScript-приложений, появившийся в 2012 году. Webpack решает специфическую для фронтенда задачу: он объединяет множество JavaScript-модулей, CSS-файлов, изображений и других ресурсов в оптимизированные бандлы для доставки в браузер. Среди возможностей Webpack — code splitting (разделение кода на части, загружаемые по требованию), tree shaking (удаление неиспользуемого кода) и hot module replacement (обновление модулей без перезагрузки страницы при разработке).

Сравнительная характеристика популярных систем сборки:

Инструмент	Экосистема	Особенности	Применение
Gradle	Java, Kotlin, Android	DSL на Groovy/Kotlin, инкрементальная сборка	Android-приложения, корпоративные JVM-проекты
Maven	Java	XML-конфигурация, соглашения вместо настроек	Классические Java-проекты, библиотеки

Инструмент	Экосистема	Особенности	Применение
Bazel	Мультиязычный	Воспроизводимые сборки, распределённое кеширование	Крупные монорепозитории (Google, Uber)
Webpack	JavaScript, TypeScript	Бандлинг модулей, обработка активов	Фронтенд-приложения, SPA
Vite	JavaScript, TypeScript	ESM, быстрый dev-сервер, Rollup для продакшена	Современные фронтенд-проекты (Vue, React)

4. ЭТАП ТЕСТИРОВАНИЯ (TEST)

Тестирование — процесс оценки программного продукта с целью выявления дефектов и подтверждения соответствия требованиям. В рамках непрерывной интеграции тестирование выполняется автоматически после каждой успешной сборки, что позволяет быстро обнаруживать регрессии и поддерживать стабильность кодовой базы.

Классификация тестов по уровню детализации:

Модульные тесты (unit tests) проверяют корректность отдельных функций, методов или классов в изоляции от остальной системы. Они выполняются быстро (миллисекунды на тест), легко поддаются автоматизации и позволяют точно локализовать дефекты. Для изоляции тестируемого кода от внешних зависимостей используются заглушки (stubs) и имитаторы (mocks).

Интеграционные тесты (integration tests) проверяют взаимодействие нескольких компонентов между собой: корректность вызовов между модулями, работу с базой данных, общение с внешними сервисами. Они медленнее модульных, но выявляют ошибки интеграции, невидимые на уровне отдельных единиц кода.

Системные тесты (end-to-end, E2E) проверяют поведение приложения целиком — от пользовательского интерфейса до базы данных. E2E-тесты наиболее приближены к реальному сценарию использования, но одновременно являются самыми медленными и хрупкими.

Помимо функциональных, применяются и другие виды тестов: нагрузочные (оценка производительности под нагрузкой), тесты безопасности (SAST, DAST), тесты доступности (accessibility), визуальные регрессионные тесты и многие другие. В современных проектах также широко применяются автоматические проверки качества кода — статический анализ (SonarQube, ESLint), анализ покрытия тестами (coverage), проверка стиля.

5. ПИРАМИДА ТЕСТИРОВАНИЯ

Пирамида тестирования — концепция, предложенная Майком Коном в 2009 году и ставшая классикой современного подхода к тестированию. Она предписывает поддерживать определённое соотношение между количеством тестов разных уровней: наибольшая часть — это быстрые модульные тесты, средняя — интеграционные, а наименьшая — медленные E2E-тесты.

Логика пирамиды основана на балансе между скоростью выполнения, стоимостью написания и поддержки, а также уровнем достоверности результатов. Типичное соотношение количества тестов выглядит следующим образом:

- Unit-тесты — около 70% от общего числа, выполняются за секунды, дешёвы в написании;
- Интеграционные тесты — около 20%, выполняются за минуты, требуют поднятия инфраструктуры;
- E2E-тесты — около 10%, выполняются за десятки минут, сложны в поддержке, но дают максимальное доверие.

Нарушение пропорций пирамиды приводит к характерным антипаттернам. «Мороженое» (ice cream cone) — перевёрнутая пирамида с

преобладанием ручных и E2E-тестов — типично для проектов, в которых автоматизация началась поздно. «Песочные часы» (hourglass) — много unit- и E2E-тестов при нехватке интеграционных — встречается, когда команда не уделяет внимания интеграционным сценариям, из-за чего ошибки взаимодействия модулей обнаруживаются только в E2E-тестах или уже в продакшене.

Test-Driven Development (TDD) — практика, тесно связанная с модульным тестированием. Она предполагает написание теста до реализации функциональности по циклу «красный-зелёный-рефакторинг»: сначала пишется падающий тест (красный), затем — минимальный код, проходящий этот тест (зелёный), и только потом код улучшается с сохранением зелёного статуса (рефакторинг). TDD способствует созданию хорошо тестируемой архитектуры и высокой степени покрытия.

6. ИНСТРУМЕНТЫ ТЕСТИРОВАНИЯ: JUNIT, JEST, PLAYWRIGHT

JUnit — стандартный фреймворк модульного тестирования для Java, созданный Кентом Беком и Эрихом Гаммой в 1997 году. JUnit заложил основы всего семейства xUnit-фреймворков, получивших распространение во многих языках программирования. Текущая версия JUnit 5 (Jupiter) поддерживает параметризованные тесты, динамические тесты, параллельное выполнение и расширяемую архитектуру.

Пример простого JUnit-теста:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    @Test
    void additionShouldReturnCorrectSum() {
        Calculator calc = new Calculator();
```

```
        assertEquals(5, calc.add(2, 3));
    }
}
```

Jest — фреймворк тестирования для JavaScript, разработанный компанией Facebook (ныне Meta) в 2014 году для тестирования React-приложений. Jest объединяет в одном пакете test runner, библиотеку ассертов, систему моков и инструменты сбора покрытия. Его отличительные особенности — концепция snapshot-тестирования (автоматическое сохранение и сравнение структур данных), нулевая конфигурация для большинства проектов и изолированное выполнение тестов в параллельных процессах.

Playwright — современный фреймворк end-to-end тестирования веб-приложений, разработанный Microsoft и выпущенный в 2020 году. Playwright позиционируется как преемник более старых инструментов (Selenium, Cypress) и предлагает ряд существенных преимуществ: поддержку всех основных браузеров (Chromium, Firefox, WebKit) через единый API, автоматическое ожидание элементов, встроенную запись видео и трейсов, возможность тестирования мобильных версий через эмуляцию устройств.

Пример E2E-теста на Playwright:

```
import { test, expect } from '@playwright/test';

test('user can log in', async ({ page }) => {
  await page.goto('https://app.example.com/login');
  await page.fill('[name=email]', 'user@test.com');
  await page.fill('[name=password]', 'secret123');
  await page.click('button[type=submit]');
  await expect(page).toHaveURL(/dashboard/);
});
```

Jenkins и Buildkite — специализированные CI-серверы, организующие запуск тестов в рамках сборочного конвейера. Jenkins, существующий с 2011 года, остаётся одним из самых популярных решений благодаря развитой экосистеме из более чем 1800 плагинов, обеспечивающих интеграцию с практически любыми инструментами. Buildkite — более современное коммерческое решение, отличающееся гибридной архитектурой: сама платформа работает в облаке, а агенты для выполнения заданий клиент устанавливает на собственную инфраструктуру, что совмещает удобство SaaS с контролем над средой выполнения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения настоящей работы были рассмотрены ключевые аспекты этапов сборки и тестирования в рамках концепции осознанной интеграции. Установлено, что эти этапы формируют ядро непрерывной интеграции и во многом определяют скорость и качество работы команды разработки.

Этап сборки превращает исходный код в готовые к развёртыванию артефакты и требует применения специализированных инструментов, оптимизированных под конкретные технологические экосистемы: Gradle для проектов на JVM, Bazel для крупных мультиязычных монорепозиториев, Webpack для современных веб-приложений. Ключевыми характеристиками современной сборки являются инкрементальность, воспроизводимость и интеграция с системами контроля версий GitHub и GitLab.

Этап тестирования, построенный в соответствии с концепцией пирамиды, обеспечивает многоуровневую проверку качества: от быстрых модульных тестов с помощью JUnit и Jest до комплексных E2E-проверок на Playwright. Осознанный подход к тестированию предполагает разумный баланс между различными уровнями, применение практики TDD и интеграцию с CI-серверами Jenkins и Buildkite.

Таким образом, осознанная интеграция — это не просто набор технических решений, а целостная инженерная культура, требующая от команды понимания целей, разумного выбора инструментов и постоянного совершенствования процессов. Именно такой подход позволяет современным командам разработки сохранять высокую скорость выпуска изменений без ущерба для качества продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Фаулер М. Рефакторинг. Улучшение существующего кода. — СПб.: Символ-Плюс, 2017. — 432 с.
2. Бек К. Экстремальное программирование: разработка через тестирование. — СПб.: Питер, 2017. — 224 с.
3. Кон М. Scrum: гибкая разработка ПО. — М.: Вильямс, 2011. — 576 с.
4. Хамбл Дж., Фарли Д. Непрерывное развёртывание ПО: автоматизация процессов сборки, тестирования и внедрения новых версий программ. — М.: Вильямс, 2011. — 432 с.
5. Ким Дж., Дебуа П., Уиллис Дж., Хамбл Дж. Руководство по DevOps. — М.: Манн, Иванов и Фербер, 2018. — 416 с.
6. Официальная документация Gradle [Электронный ресурс]. — URL: https://docs.gradle.org (дата обращения: 24.04.2026).
7. Официальная документация Bazel [Электронный ресурс]. — URL: https://bazel.build/docs (дата обращения: 24.04.2026).
8. Официальная документация Webpack [Электронный ресурс]. — URL: https://webpack.js.org (дата обращения: 24.04.2026).
9. Документация JUnit 5 [Электронный ресурс]. — URL: https://junit.org/junit5/docs (дата обращения: 24.04.2026).
10. Документация Jest [Электронный ресурс]. — URL: https://jestjs.io/docs (дата обращения: 24.04.2026).

11. Документация Playwright [Электронный ресурс]. — URL: https://playwright.dev/docs (дата обращения: 24.04.2026).